

มหาวิทยาลัยราชภัฏบุรีรัมย์ — BRU Strategic Tracking System

โครงสร้างระบบ

อธิบายโฟลเดอร์ทั้งหมด พร้อมหน้าตาการทำงาน (Folder Structure & How It Works)

ฉบับสมบูรณ์

อธิบายโฟลเดอร์ทั้งหมดของระบบ BRU Strategic Tracking System

พร้อมหน้าตาการทำงาน (What Each Folder Does & How It Works)

🌳 แผนผังโครงสร้างโฟลเดอร์ทั้งหมด

c:\St_bru\	← Root ของโปรเจกต์ทั้งหมด
├── backend/	← ผังเซิร์ฟเวอร์ (Node.js + Express)
│ ├── config/	← ตั้งค่าการเชื่อมต่อภายนอก
│ ├── middleware/	← ด้านตรวจสอบก่อนเข้า Controller
│ ├── routes/	← กำหนดเส้นทาง URL → Controller
│ ├── controllers/	← Logic หลักทั้งหมดของระบบ
│ ├── prisma/	← จัดการฐานข้อมูลและ Seed Data
│ ├── uploads/	← เก็บไฟล์รูปภาพหลักฐานกิจกรรม
│ └── app.js	← จุดเริ่มต้นเซิร์ฟเวอร์
├── frontend/src/	← ผังหน้าจอ (React + Vite)
│ ├── pages/	← หน้าจอแยกตาม Role
│ │ ├── auth/	← หน้า Login
│ │ ├── teacher/	← หน้าจอสําหรับอาจารย์
│ │ ├── dean/	← หน้าจอสําหรับคณบดี
│ │ ├── president/	← หน้าจอสําหรับอธิการบดี
│ │ ├── admin/	← หน้าจอสําหรับแอดมิน
│ │ └── executive/	← หน้าจอรายละเอียดสําหรับผู้บริหาร
│ ├── components/	← ชิ้นส่วน UI ที่ใช้ร่วมกัน
│ ├── layouts/	← โครงกระดูกหน้าจอ (Topbar + Sidebar)
│ ├── contexts/	← เก็บสถานะผู้ใช้งานระดับระบบ
│ ├── services/	← ตัวกลางเชื่อมต่อ API
│ └── utils/	← ฟังก์ชันช่วยเหลือทั่วไป
├── database/	← สคริปต์ SQL และเอกสารฐานข้อมูล
└── postman/	← ชุดทดสอบ API

● BACKEND — ฝั่งเซิร์ฟเวอร์

📁 `backend/config/` — ตั้งค่าการเชื่อมต่อภายนอก

ไฟล์	หน้าที่	หน้าที่การทำงาน
<code>prisma.js</code>	สร้าง PrismaClient ตัวเดียวของระบบ (Singleton)	เปิดตัวเซิร์ฟเวอร์ครั้งแรก → ต่อ MySQL → ทุก Controller ใช้ตัวเดียวกัน
<code>cloudinary.js</code>	ตั้งค่าเชื่อมต่อ Cloudinary (CDN รูปภาพ)	กรณีเปิดใช้งาน Cloud Storage → อัปโหลดรูปขึ้น Cloudinary แทน Local

ลำดับการทำงาน:

app.js เริ่มต้น → โหลด prisma.js → ต่อ MySQL → พร้อมให้ Controller ใช้งาน

📁 `backend/middleware/` — ด้านตรวจสอบ (3 ไฟล์)

แนวคิด: ทุก API Request ต้องผ่านด่านเหล่านี้ก่อน ถึงจะเข้า Controller ได้

ไฟล์	หน้าที่	หน้าที่การทำงาน
<code>auth.middleware.js</code>	ตรวจ JWT Token + คุณสมบัติตาม Role (RBAC)	เปิด API → ตรวจ Token → ตรวจ Role → ผ่าน/ปฏิเสธ
<code>upload.middleware.js</code>	รับไฟล์รูปภาพ ตรวจสอบนามสกุล จำกัดขนาด ≤5MB	ฟอร์มส่งรูป → Multer รับ → ตรวจ .jpg/.png/.webp → บันทึก /uploads
<code>validation.middleware.js</code>	ตรวจสอบความถูกต้องของข้อมูลที่ส่งมา	ส่งฟอร์ม → Validator ตรวจ → ผิดรูปแบบ → แจ้ง Error ก่อนบันทึก DB

ลำดับการทำงานจริง:

Request → auth.middleware → validation.middleware → Controller → Response
(ถ้าเป็น upload ก็ผ่าน upload.middleware ด้วย)

📁 `backend/routes/` — กำหนดเส้นทาง URL (8 ไฟล์)

แนวคิด: Routes เป็นสารบัญบอกว่า URL ไหน ให้ไปทำงานที่ฟังก์ชันใดใน Controller

ไฟล์	URL Prefix	หน้าที่
<code>auth.routes.js</code>	<code>/api/auth</code>	Login, ดูโปรไฟล์, เปลี่ยนรหัสผ่าน, อัปโหลดรูปโปรไฟล์
<code>master.routes.js</code>	<code>/api/master</code>	CRUD Master Data 9 หมวด (ยุทธศาสตร์, คณะ, ปีงบประมาณ)
<code>project.routes.js</code>	<code>/api/projects</code>	สร้าง/แก้ไข/ลบ/ดูโครงการ, ล็อกแผนงาน
<code>activity.routes.js</code>	<code>/api/activities</code>	บันทึกกิจกรรม, อัปโหลดรูปหลักฐาน, ลบกิจกรรม
<code>dashboard.routes.js</code>	<code>/api/dashboard</code>	สถิติแดชบอร์ดตาม Role (Teacher/Dean/President)

ไฟล์	URL Prefix	หน้าที่
<code>directive.routes.js</code>	<code>/api/directives</code>	ส่งข้อสั่งการ Dean/President ไปยังโครงการ
<code>report.routes.js</code>	<code>/api/reports</code>	รายงานสรุปงบประมาณและผลผลิตตามยุทธศาสตร์
<code>issue.routes.js</code>	<code>/api/issues</code>	แจ้งปัญหาระบบ, Admin อัปเดตสถานะ

📁 `backend/controllers/` – Logic หลักของระบบ (7 ไฟล์)

แนวคิด: Controller คือสมองของระบบ ประมวลผลธุรกิจ คำนวณ และสั่ง Prisma ดึง/บันทึกข้อมูล

ไฟล์	ขนาด	หน้าที่ + สิ่งที่ทำในระบบ
<code>auth.controller.js</code>	12KB	Login (Bcrypt + JWT), ดูโปรไฟล์, เปลี่ยนรหัสผ่าน, สร้างรหัสบุคลากร 6 หลัก
<code>project.controller.js</code>	26KB	CRUD โครงการ, คำนวณ % Progress Rollup, ระบบล็อกแผนงาน, ตรวจสอบสิทธิ์ IDOR
<code>activity.controller.js</code>	19KB	บันทึกกิจกรรม, อัปเดตรูปหลักฐาน, คำนวณยอดงบประมาณ
<code>dashboard.controller.js</code>	41KB	สถิติ Teacher/Dean/President Dashboard, ตรวจจับ Red Flag (MBE)
<code>master.controller.js</code>	24KB	CRUD ยุทธศาสตร์ 4 ระดับ, คณะ, สาขา, ปีงบประมาณ, แหล่งงบประมาณ
<code>report.controller.js</code>	37KB	รายงานสรุปงบประมาณและผลผลิตแยกตามยุทธศาสตร์ทุกระดับ
<code>issue.controller.js</code>	11KB	รับแจ้งปัญหา บันทึกรายละเอียด Admin อัปเดตสถานะ

📁 `backend/prisma/` – จัดการฐานข้อมูล

ไฟล์	หน้าที่	เมื่อใช้
<code>schema.prisma</code>	แบบจำลองตาราง 14 ตาราง ความสัมพันธ์ และ Enum Role	ตอนออกแบบ/แก้ไขโครงสร้าง DB
<code>seed.js</code>	สร้างข้อมูลเริ่มต้นครบทุกส่วน (Master Data + User ทุก Role + โครงการ ตัวอย่าง)	ติดตั้งระบบครั้งแรก
<code>seed-admin-only.js</code>	สร้างเฉพาะ User Admin ตัวเดียว	Reset ระบบ
<code>seed-strategic-plan.js</code>	ใส่ข้อมูลยุทธศาสตร์ 4 ระดับ	เตรียมข้อมูลยุทธศาสตร์
<code>migrations/</code>	ประวัติการเปลี่ยนแปลงโครงสร้าง DB	อัตโนมัติโดย Prisma

📁 `backend/uploads/` – ไฟล์รูปภาพหลักฐาน

```
uploads/
├── activities/  ← รูปภาพหลักฐานกิจกรรมลงพื้นที่จริง (.jpg, .png, .webp)
└── avatars/    ← รูปโปรไฟล์ของพนักงาน
```

เมื่ออาจารย์อัปโหลดรูปหลักฐาน → Multer รับไฟล์ → บันทึกลง `uploads/activities/` → URL ถูกบันทึกลง DB → แสดงผลในหน้ารายละเอียดโครงการ

FRONTEND — ฝั่งหน้าจอ (React)

📁 `frontend/src/pages/` — หน้าจอแยกตาม Role (6 ไฟล์)

📁 `auth/` — หน้าเข้าสู่ระบบ

ไฟล์	หน้าที่การทำงาน
Login.jsx	หน้าล็อกอิน → กรอก username/password → ส่ง POST /api/auth/login → รับ JWT Token → บันทึก Context → redirect ไปแดชบอร์ดตาม Role

📁 `teacher/` — หน้าจอสำหรับ**อาจารย์** (6 ไฟล์)

ไฟล์	หน้าที่การทำงาน
Dashboard.jsx	แดชบอร์ดส่วนตัว: สรุปโครงการที่ตนรับผิดชอบ, งบประมาณ, Progress รวม, รูปภาพล่าสุด
Projects.jsx	ตารางรายการโครงการทั้งหมดที่ตนเองดูแล, ค้นหา, กรองสถานะ, กดเปิดดูรายละเอียด
ProjectForm.jsx	ฟอร์มสร้าง/แก้ไขโครงการ — Cascading Dropdown ยุทธศาสตร์ 4 ระดับ, เลือกทีมงาน, กำหนดงบ/เป้าหมาย
ProjectDetails.jsx	รายละเอียดโครงการ — แสดง Progress Bar, กิจกรรมย่อยทั้งหมด, รูปหลักฐาน, ดูข้อสั่งการผู้บริหาร
ActivitiesList.jsx	รายการกิจกรรมย่อยทั้งหมด, บันทึกผลผลิตจริง, อัปโหลดรูปภาพ
Gallery.jsx	แกลเลอรี่รูปภาพหลักฐานกิจกรรมทั้งหมดของตนเอง

📁 `dean/` — หน้าจอสำหรับ**คณบดี** (3 ไฟล์)

ไฟล์	หน้าที่การทำงาน
Dashboard.jsx	แดชบอร์ดภาพรวมคณะ: สถิติงบประมาณ, กราฟเปรียบเทียบสาขา (Cross-Dept Matrix), โครงการ Red Flag, พิมพ์ข้อสั่งการ
Projects.jsx	รายการโครงการทุกโครงการในคณะ (Data Isolation — เห็นเฉพาะคณะตัวเอง)
Reports.jsx	รายงานสรุปผลงานและการเบิกจ่ายงบประมาณรายสาขาวิชาภายในคณะ

📁 `president/` — หน้าจอสำหรับ**อธิการบดี** (3 ไฟล์)

ไฟล์	หน้าที่การทำงาน
Dashboard.jsx	Strategic Dashboard ภาพรวมทั้งมหาวิทยาลัย: Strategic Heatmap, Red Flags ทุกคณะ, สั่งการโครงการวิกฤต
Projects.jsx	รายการโครงการทุกโครงการทั่วมหาวิทยาลัย (ดูได้ทั้งหมด ไม่มี Isolation)
Reports.jsx	รายงานสรุปผลงานเทียบยุทธศาสตร์ระดับมหาวิทยาลัย

📁 `admin/` — หน้าจอสำหรับ**แอดมิน** (6 ไฟล์)

ไฟล์	หน้าที่การทำงาน
Dashboard.jsx	แดชบอร์ดภาพรวมระบบ: ข้อมูลทุกคณะ, สถิติรวม, รูปกิจกรรมล่าสุด
MasterData.jsx	จัดการ Master Data 9 หมวด: ยุทธศาสตร์ 4 ระดับ, ینگบฯ, แหล่งงบฯ, คณะ, สาขา, จัดการ User

ไฟล์	หน้าที่การทำงาน
Projects.jsx	รายการโครงการทุกโครงการในระบบ ลบ/แก้ไขข้ามคณะได้
AdminActivities.jsx	กำกับติดตามกิจกรรมทุกโครงการทั่วประเทศ
Reports.jsx	รายงานรวมทุกระดับ Export ได้
Issues.jsx	รับเรื่องแจ้งปัญหา อัปเดตสถานะ (Pending/In Progress/Resolved)

📁 `executive/` — หน้าจอรายละเอียดสำหรับผู้บริหาร** (1 ไฟล์)**

ไฟล์	หน้าที่การทำงาน
ProjectDetail.jsx	Modal/หน้ารายละเอียดโครงการแบบผู้บริหาร: ดู Progress, รูปหลักฐาน, พิมพ์ข้อสั่งการ Dean/President ส่งตรงถึงอาจารย์

📁 `frontend/src/components/` — ชิ้นส่วน UI ที่ใช้ร่วมกัน (7 ไฟล์)

ไฟล์	หน้าที่การทำงาน
Topbar.jsx	แถบด้านบน: ชื่อผู้ใช้, Role Badge, ปุ่มแจ้งปัญหา, ปุ่ม Logout
Sidebar.jsx	เมนูด้านซ้าย: แสดงเมนูตาม Role ที่ล็อกอิน (TEACHER เห็นคนละเมนูกับ DEAN)
ExecutiveProjectModal.jsx	Popup รายละเอียดโครงการสำหรับ Dean/President พร้อมช่องพิมพ์คำสั่งการ
ReportIssueModal.jsx	Popup ฟอรัมแจ้งปัญหาระบบ (ทุก Role กดแจ้งได้)
ProfileModal.jsx	Popup ดูและแก้ไขข้อมูลโปรไฟล์, อัปโหลดรูปโปรไฟล์
CustomSelect.jsx	Dropdown แบบ Custom ที่รองรับ Search ในตัว (ใช้ใน Cascading ยุทธศาสตร์)
ErrorBoundary.jsx	ดัก Error ระดับ Component ป้องกันทั้งระบบล่มเมื่อเกิด JavaScript Error

📁 `frontend/src/layouts/` — โครงกระดูกหน้าจอ

ไฟล์	หน้าที่การทำงาน
AppLayout.jsx	กรอบหลักทุกหน้า: จัดวาง Topbar (บน) + Sidebar (ซ้าย) + Main Content (ขวา) — ทุกหน้าจะอยู่ภายใน Layout นี้



📁 `frontend/src/contexts/` — เก็บสถานะผู้ใช้งานระดับระบบ

ไฟล์	หน้าที่การทำงาน
AuthContext.jsx	เก็บข้อมูลผู้ใช้ที่ล็อกอินอยู่ (User, Role, Token) ให้ทุกหน้าเรียกใช้ได้ ไม่ต้องส่งผ่าน props ทีละชั้น

ลำดับการทำงาน:

Login สำเร็จ → บันทึกใน AuthContext → ทุก Component ดึงข้อมูล User ได้ทันที

📁 `frontend/src/services/` — ตัวกลางเชื่อมต่อ API

ไฟล์	หน้าที่การทำงาน
<code>api.js</code>	ตั้งค่า Axios Instance กำหนด Base URL ของ Backend, แนบ JWT Token ใน Header ทุก Request อัตโนมัติ

```
// ทุก API Call ในระบบจะผ่านตัวนี้ → แนบ Token ให้อัตโนมัติ
axios.defaults.headers['Authorization'] = `Bearer ${token}`;
```

📁 `frontend/src/utils/` — ฟังก์ชันช่วยเหลือ

ไฟล์	หน้าที่การทำงาน
<code>imageUrl.js</code>	แปลง Path รูปภาพจาก Server ให้เป็น URL สมบูรณ์ที่แสดงผลได้ (รองรับทั้ง Local + Cloudinary)
<code>statusHelper.js</code>	แปลงสถานะโครงการ (GREEN/AMBER/RED) เป็นสีและข้อความภาษาไทยที่แสดงบนหน้าจอ



DATABASE — ฐานข้อมูล

 `database/` — สคริปต์และเอกสาร SQL (4 ไฟล์)

ไฟล์	หน้าตาการทำงาน
<code>schema.sql</code>	สคริปต์ SQL สร้างตาราง 14 ตารางบน MySQL (ใช้สำหรับสร้าง DB โดยตรง)
<code>seed.sql</code>	สคริปต์ INSERT ข้อมูลเริ่มต้น (Master Data + User ตัวอย่าง)
<code>DataDictionary.md</code>	เอกสารอธิบายทุกฟิลด์ในทุกตาราง (ชนิดข้อมูล, ความหมาย, ตัวอย่าง)
<code>ERD.md</code>	ER Diagram แสดงความสัมพันธ์ระหว่างตาราง



POSTMAN – ชุดทดสอบ API



`postman/` – (3 ไฟล์)

ไฟล์	หน้าตาการทำงาน
BRU_ ... _Collection.json	ชุดทดสอบ API ครบ 8 หมวดหมู่ กว่า 25+ Requests พร้อม Script เก็บ Token อัตโนมัติ
BRU_ ... _Environment.json	ตัวแปรสภาพแวดล้อม: Base URL, Token, บัญชีผู้ใช้ 4 Role
README.md	คู่มือ Import และทดสอบ API ผ่าน Postman ใน 2 คลิก

สรุปลำดับการทำงานของโฟลเดอร์ทั้งหมด

